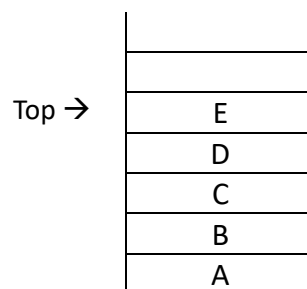


Stack Data Structure

A Stack is an ordered collection of items into which new items may be inserted and from which items may be inserted and from which items may be deleted at one end, called the top of the stack.

New items may be put on top of the stack (in which case the top of the stack moves upward to correspond to the new highest element), or items which are at the top of the stack may be removed (in which case the top of the stack moves downward to correspond to the new highest element).

Example:



Stack containing elements

The stack with some elements A,B,C,D,E is as shown above. E is at the top of the stack. If we want to remove any element from the stack, first we have to remove the top element E. Then the top of the stack will be pointing to D and so on.

Suppose we want to remove (pop) an element E, after removing the element top is decremented by 1 and points to next element D.

Suppose we want to add (push) an element to the stack, first the top is incremented, then the element is pushed onto the stack.

Primitive operations on Stack:

PUSH and POP are the two primitive operations of Stack.

When an item is added to a stack, it is pushed onto the stack. The operation is PUSH.

When an item needs to be removed from the stack, it is popped from the stack. The operation is POP.

Suppose given a stack S and an item x performing the operation `push(&s, x)` adds the item x to the stack S.

Similarly, the operation `pop(&s)` removes the top element and returns it as a function value.

Thus the assignment statement, $x = \text{pop}(\&s)$; removes the element at the top of the stack S and assign its value to x .

Because of the push operation adds elements to the stack, a stack is some times called a pushdown list.

If a stack contains a single item and the stack is popped, the resulting stack contains no items called the empty stack.

There for, before applying the pop operator to a stack we must ensure that the stack is not empty. The result of an illegal attempt to pop or access an item from an empty stack is called underflow.

Consider a mathematical expression that includes several parentheses

$$7 - ((x * ((x + y) / (j - 3)) + y) / (4 - 2.5))$$

We have to ensure that the parentheses are nested correctly.

So we have to check following two conditions:

1. There are equal number of right and left parentheses.
2. Every right parenthesis is preceded by a matching left parentheses.

Expression $((A+B)$ violates condition1.

Expression $)A+B(-C$ violates condition 2.

To solve the problem, define the parentheses count at a particular point in an expression as the no. of left parentheses minus the no. of right parentheses that have been encountered in scanning the expression from its left end up to that particular point.

1. If the parentheses count at the end of the expression is zero, then equal no. of right parentheses and left parentheses.
2. The parentheses count at each point in the expression is non-negative.
(No right parenthesis is encountered for which a matching left parenthesis had not previously been encountered.)

$$7 - ((x * ((x + y) / (j - 3)) + y) / (4 - 2.5))$$

Parentheses count : 0 0 1 2 2 2 3 4 4 4 4 3 3 4 4 4 4 3 2 2 2 1 1 2 2 2 2 1 0

Now suppose assume that there are different types of scope delimiters exist such as $(, [, \}$.

In this case, the expression such as (A +B] or [(A + B]) are illegal.

Therefore, it is necessary to keep track of not only how many scopes have been opened but also of their types.

A stack may be used to keep track of the types of scopes encountered. Whenever a scope opener is encountered it is pushed on to the stack. Whenever a scope ender is encountered, stack is examined. If the stack is empty, the scope ender does not have a matching opener, the expression is therefore invalid.

If the stack is not empty, we pop the stack and check whether the popped item corresponds to the scope ender. If a match occurs, we continue. If it does not, the expression is invalid. When the end of the string is reached, the stack must be empty.

The algorithm is given below:

Start

```
valid = True /* assume the expression is valid initially*/
S = empty stack
while( we have not read the entire string)
{
    read the next symbol of the string;
    if( symbol == '(' || symbol == '[' || symbol == '{')
        push(&S, symbol);
    if( symbol == ')' || symbol == ']' || symbol == '}')
    {
        if( empty(&s) /* stack is empty */
        valid = false;
        else
        {
            i = pop(&s);
            if (i is not the matching opener of symbol)
                valid = false;
        }
    }
}
if(!empty(&s) /* if stack is not empty */
    valid = false;
if (valid) /* if valid is true */
    printf("The expression is valid ");
else
    printf("The expression is invalid ");
end.
```